



ETUDE DES GRAPHS

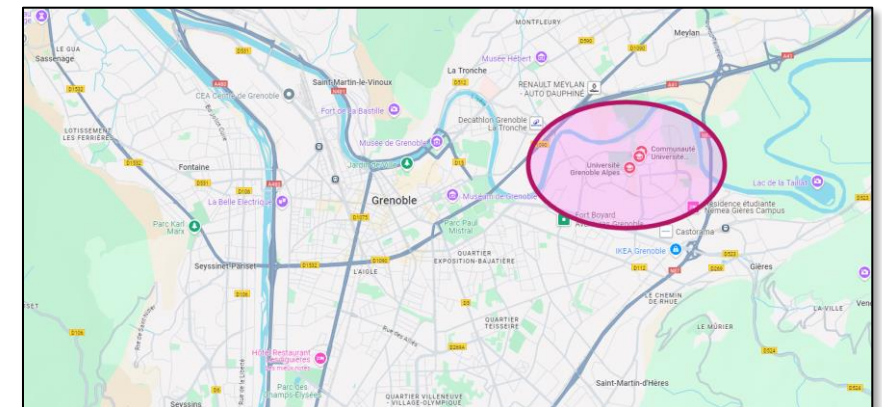
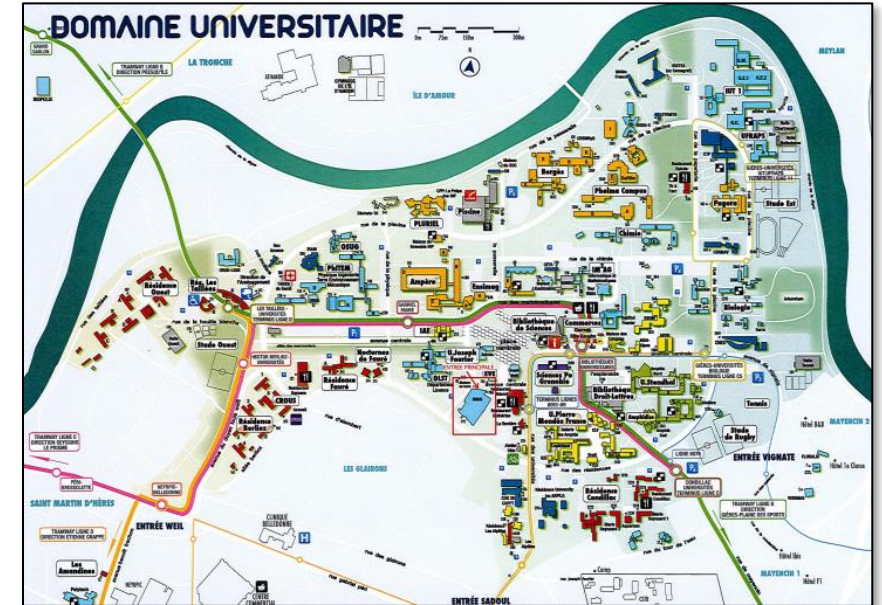
Parcours d'un Campus Universitaire
(Domaine Universitaire de Grenoble)

PRÉSENTATION

PRÉSENTATION

- **Présentation du campus :**

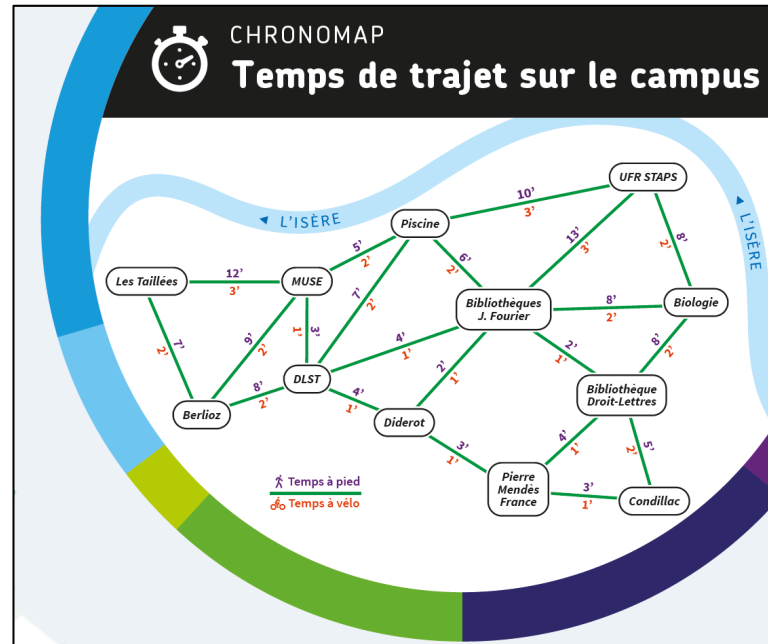
- Le domaine/campus de la ville de Grenoble est localisé dans une ancienne zone marécageuse de deux villes limitrophes à Grenoble (Gières et St Martin d'Hères).
- Ses principales caractéristiques :
 - 35 000 étudiants
 - 3 000 enseignants
 - Superficie de 186 hectares
 - 3 lignes de tramways
 - Pistes cyclables et sentiers permettant le déplacement pédestre ou à vélo.



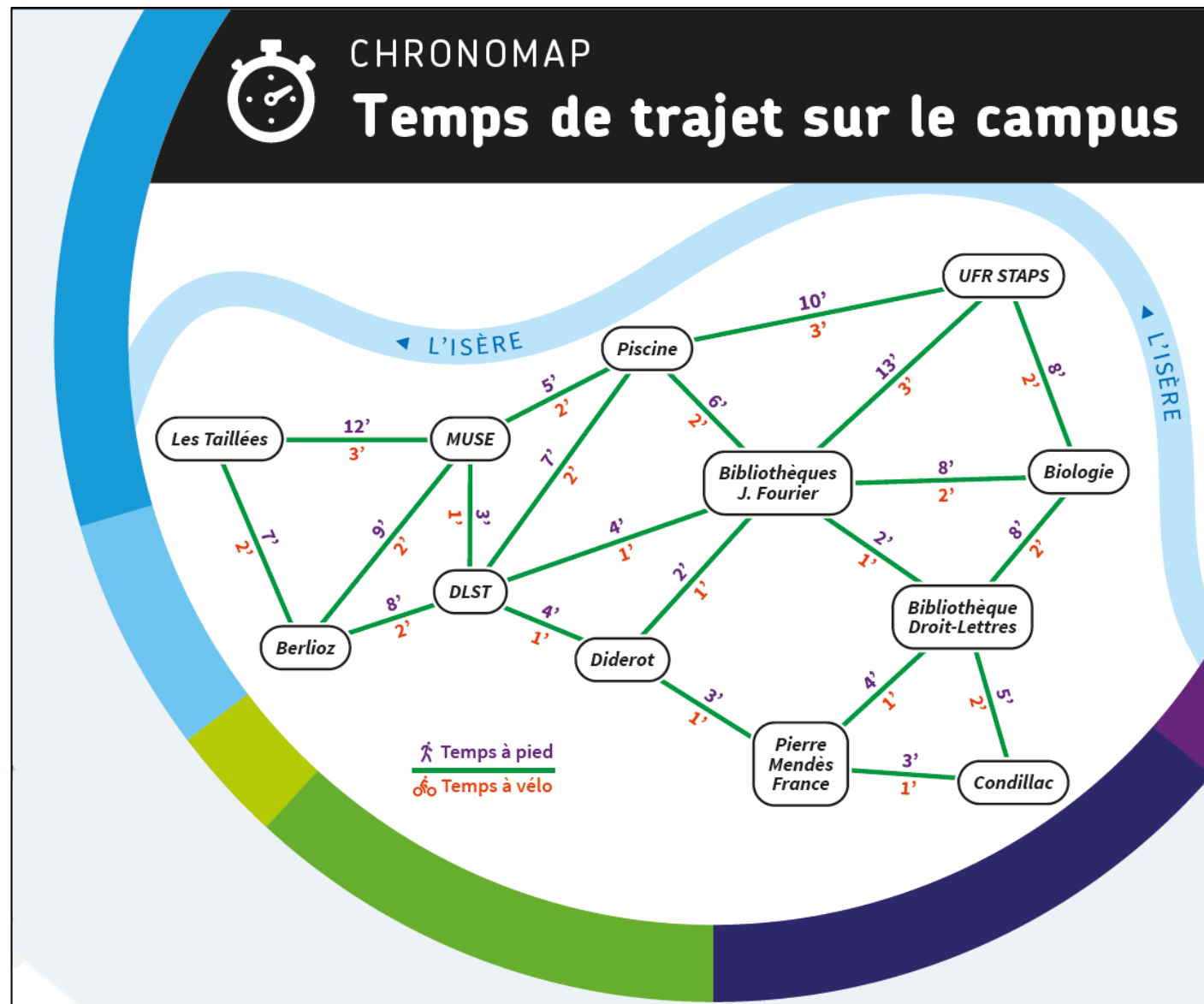
PRÉSENTATION

- **La chronomap**

- Afin de faciliter et promouvoir les déplacements sur le campus, une carte a été réalisée.
- Cette carte indique les temps de déplacements à pied et en vélo.



PRÉSENTATION



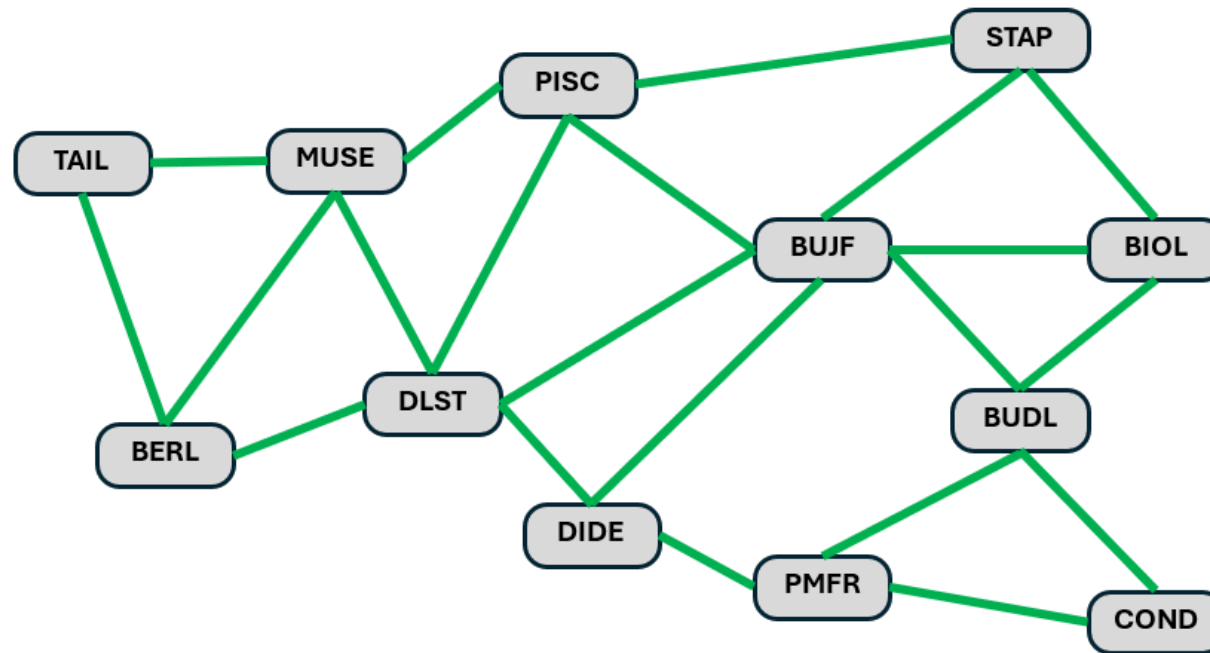
TRAVAIL À RÉALISER

1/ Etude sans prise en compte des pondérations

1/ ETUDE SANS PRISE EN COMPTE DES PONDÉRATIONS

- **Modélisation sous forme de graphe :**

- Pour cette 1ère partie, nous vous proposons de ne pas prendre en compte la durée de traversé des arêtes. L'ensemble des lieux sont abrégés avec 4 caractères, nous obtenus alors le graphe suivant :

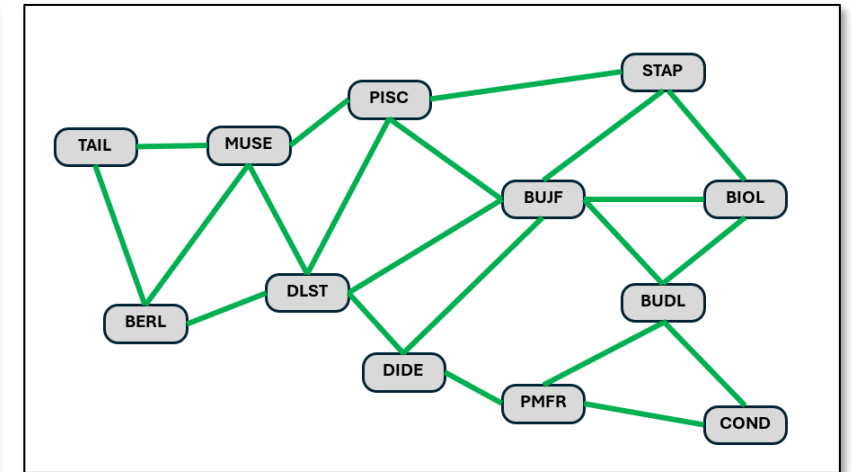


1/ ETUDE SANS PRISE EN COMPTE DES PONDÉRATIONS

- 1.1/ Liste & matrice d'adjacence

- Représenter le graphe sous la forme d'une matrice d'adjacence.

```
# BUJF | PISC | STAP | BIOL | BUDL | DIDE | DLST | MUSE | COND | PMFR | BERL | TAIL
[[ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ...], # BUJF
 [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ...], # PISC
 [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ...], # STAP
 [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ...], # BIOL
 [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ...], # BUDL
 [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ...], # DIDE
 [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ...], # DLST
 [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ...], # MUSE
 [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ...], # COND
 [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ...], # PMFR
 [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ...], # BERL
 [ ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ..., ...]] # TAIL
```

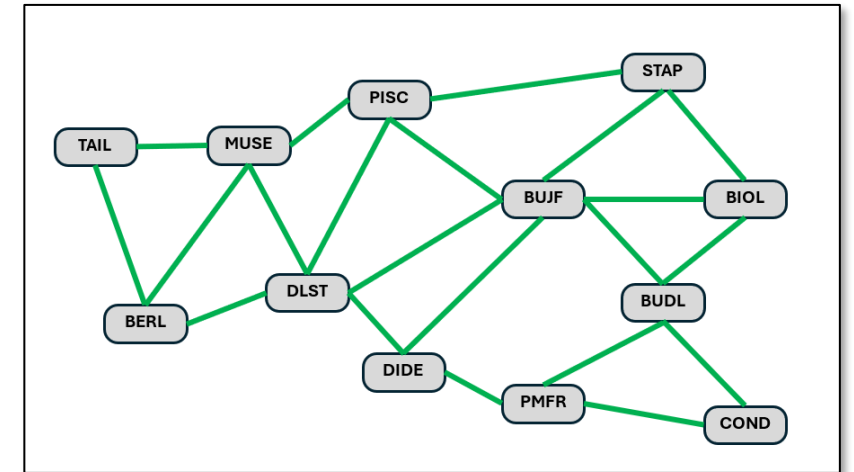


1/ ETUDE SANS PRISE EN COMPTE DES PONDÉRATIONS

- 1.1/ Liste & matrice d'adjacence

- Représenter le graphe sous la forme d'une liste d'adjacence.

```
{ "BUJF": [ ....., ....., ....., ....., ....., ..... ],  
  "PISC": [ ....., ....., ....., ..... ],  
  "STAP": [ ....., ....., ..... ],  
  "BIOL": [ ....., ....., ..... ],  
  "BUDL": [ ....., ....., ....., ..... ],  
  "DIDE": [ ....., ....., ..... ],  
  "DLST": [ ....., ....., ....., ....., ..... ],  
  "MUSE": [ ....., ....., ....., ..... ],  
  "COND": [ ....., ..... ],  
  "PMFR": [ ....., ....., ..... ],  
  "BERL": [ ....., ....., ..... ],  
  "TAIL": [ ....., ..... ] }
```



1/ ETUDE SANS PRISE EN COMPTE DES PONDÉRATIONS

- 1.2/ Parcours de l'ensemble du graphe

- L'algorithme de parcours complet en largeur du graphe est le suivant :
 - Placer le sommet de **départ** dans une **file** et une **liste** de recensement des sommets
 - **Tant que la file** n'est pas vide :
 - Prendre le **prochain sommet** à visiter dans la **file**
 - **Itérer** tous les **adjacents** du sommet visité :
 - Pour chaque sommet adjacent :
 - Le **rajouter** dans la **file**
 - Le **rajouter** dans la **liste** de recensement des sommets

1/ ETUDE SANS PRISE EN COMPTE DES PONDÉRATIONS

- 1.2/ Parcours de l'ensemble du graphe

- Compléter l'implémentation en Python de l'algorithme de parcours en largeur du graphe :

```
def parcoursLarg(graphe, depart):  
    recenses = [ ..... ]  
    file = [ ..... ]  
  
    while len(file) != .....:  
        sommetEnVisite = file.pop(0)  
        # Boucle de récupération des adjacents du sommet visité :  
        for adjacent in graphe[sommetEnVisite]:  
            if .....not in recenses:  
                file. ....  
                recenses. ....  
  
    return recenses
```


1/ ETUDE SANS PRISE EN COMPTE DES PONDÉRATIONS

- **1.3 Dictionnaire des prédécesseurs**

- Nous désirons connaître l'ensemble des chemins à partir d'un sommet de départ vers tous les autres sommets du graphe. Pour ce faire, nous allons utiliser un dictionnaire des prédécesseurs.
- Réécrire la fonction **parcoursLarg(graphe, depart)** en remplaçant la liste **recences** par un dictionnaire des prédécesseurs que nous désignerons : **dicoPredecesseurs**.

1/ ETUDE SANS PRISE EN COMPTE DES PONDÉRATIONS

- 1.3 Dictionnaire des prédécesseurs

```
def predecesseurs(graphe, depart):  
    dicoPredecesseurs = { ..... }  
    file = [ ..... ]  
  
    while len(file) != ..... :  
        sommetEnVisite = file.pop(0)  
        # Boucle de récupération des adjacents du sommet visité :  
        for adjacent in graphe[sommetEnVisite]:  
            if ..... not in dicoPredecesseurs:  
                file. ....  
                dicoPredecesseurs[ ..... ] = .....  
  
    return dicoPredecesseurs
```

1/ ETUDE SANS PRISE EN COMPTE DES PONDÉRATIONS

- **1.3 Dictionnaire des prédécesseurs**

- L'exécution de la fonction **predescesseurs(CAMP_GSMD_LSTADJ, "BUJF")** produit le résultat suivant :

```
{'BUJF': 'BUJF', 'PISC': 'BUJF', 'STAP': 'BUJF', 'BIOL': 'BUJF', 'BUDL': 'BUJF', 'DIDE': 'BUJF', 'DLST': 'BUJF', 'MUSE': 'PISC', 'COND': 'BUDL', 'PMFR': 'BUDL', 'BERL': 'DLST', 'TAIL': 'MUSE'}
```

- **En exploitant ce dictionnaire des prédécesseurs, déterminer les 2 chemins suivants :**

- BUJF à TAIL :
- BUJF à BERL :

1/ ETUDE SANS PRISE EN COMPTE DES PONDÉRATIONS

• 1.3 Dictionnaire des prédécesseurs

- La fonction **chemin(depart, fin, graphe)** ci-dessous permet de reconstituer le chemin entre un sommet **depart** et le sommet **fin**.
- Compléter les instructions manquantes de la fonction.

```
def chemin(depart, fin, graphe):  
    dicoPredecesseurs = predecesseurs(graphe, depart)  
    chemin = [ ..... ]  
    predecesseur = .....  
    while predecesseur != ..... :  
        predecesseur = dicoPredecesseurs[ ..... ]  
        chemin = [ ..... ] + .....  
    return chemin
```

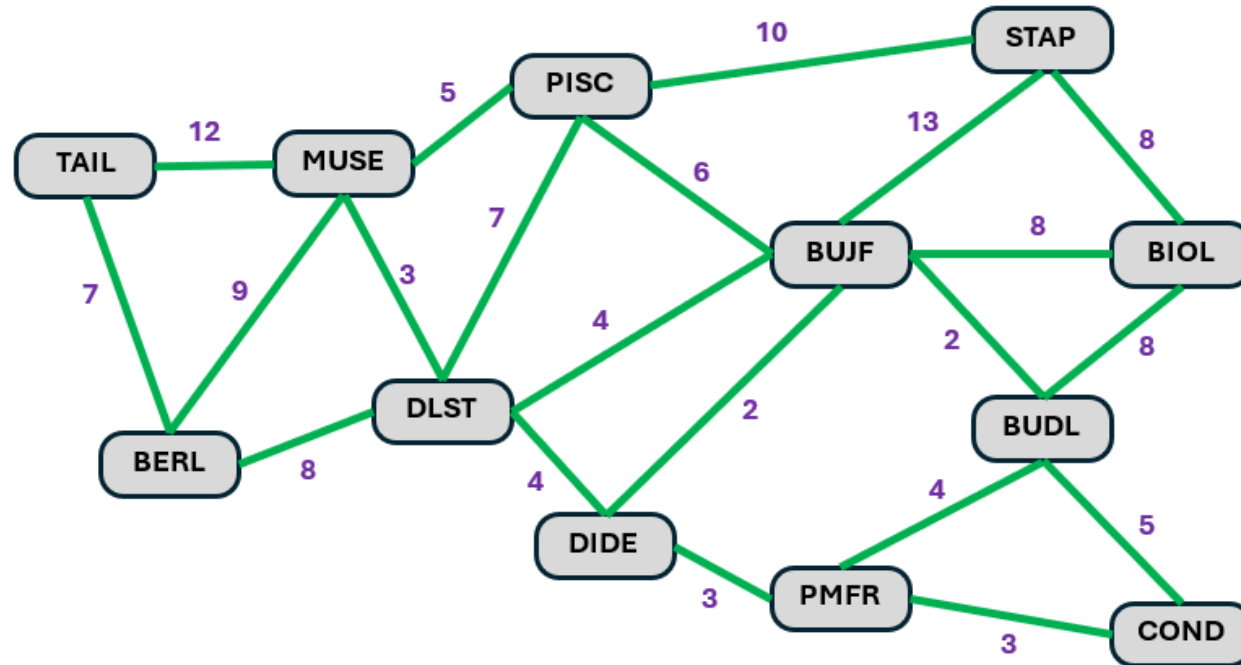

TRAVAIL À RÉALISER

2/ Etude avec prise en compte des pondérations

2/ ETUDE AVEC PRISE EN COMPTE DES PONDÉRATIONS

- **2.1/ Liste & matrice d'adjacence**

- Représenter le graphe sous la forme :
 - D'une matrice d'adjacence pondérée
 - D'une liste d'adjacence pondérée



2/ ETUDE AVEC PRISE EN COMPTE DES PONDÉRATIONS

- 2.1/ Liste & matrice d'adjacence

#	BUJF	PISC	STAP	BIOL	BUDL	DIDE	DLST	MUSE	COND	PMFR	BERL	TAIL	
[	...	...	...	...	...	...	...	...	...	...	...	...]	# BUJF
[	...	...	...	...	...	...	...	...	...	...	...	...]	# PISC
[	...	...	...	...	...	...	...	...	...	...	...	...]	# STAP
[	...	...	...	...	...	...	...	...	...	...	...	...]	# BIOL
[	...	...	...	...	...	...	...	...	...	...	...	...]	# BUDL
[	...	...	...	...	...	...	...	...	...	...	...	...]	# DIDE
[	...	...	...	...	...	...	...	...	...	...	...	...]	# DLST
[	...	...	...	...	...	...	...	...	...	...	...	...]	# MUSE
[	...	...	...	...	...	...	...	...	...	...	...	...]	# COND
[	...	...	...	...	...	...	...	...	...	...	...	...]	# PMFR
[	...	...	...	...	...	...	...	...	...	...	...	...]	# BERL
[	...	...	...	...	...	...	...	...	...	...	...	...]	# TAIL

2/ ETUDE AVEC PRISE EN COMPTE DES PONDÉRATIONS

- **2.1/ Liste & matrice d'adjacence**

```
{ "BUJF": { . . . . , . . . . , . . . . , . . . . , . . . . , . . . . },  
  "PISC": { . . . . , . . . . , . . . . , . . . . },  
  "STAP": { . . . . , . . . . , . . . . },  
  "BIOL": { . . . . , . . . . , . . . . },  
  "BUDL": { . . . . , . . . . , . . . . },  
  "DIDE": { . . . . , . . . . , . . . . },  
  "DLST": { . . . . , . . . . , . . . . , . . . . },  
  "MUSE": { . . . . , . . . . , . . . . , . . . . },  
  "COND": { . . . . , . . . . },  
  "PMFR": { . . . . , . . . . },  
  "BERL": { . . . . , . . . . },  
  "TAIL": { . . . . , . . . . }}
```